
Autonomous Inbound Content System

Complete Developer Reference Document

For developers building the system from scratch

Project: Autonomous AI pipeline for brand presence and inbound lead generation
Client: Bitcot Technology, San Diego, CA
Author: Mouriyan Gandhi (Intern)
Guided by: Jitendra Jain
Document version: 1.0 — May 2026
Purpose: End-to-end developer specification — no prior knowledge assumed

System Summary

This system is an **8-agent autonomous content pipeline** that:

1. Monitors real-time signals (Google Trends, X/Twitter, Reddit, RSS)
2. Filters topics against Bitcot's ideal client profile (ICP)
3. Researches and fact-checks from verified sources
4. Produces SEO-optimised blog posts, LinkedIn posts, and X threads
5. Schedules and publishes via API
6. Tracks performance and feeds learnings back into the system

The system has **one human review gate** before content is published. Everything else is fully automated.

Contents

1	System Overview	4
1.1	What This System Does	4
1.2	Architecture at a Glance	4
1.3	Two Operating Modes	5
1.4	Tech Stack	5
2	Memory Layer	6
2.1	What the Memory Layer Is	6
2.2	Data Schema	6
2.3	Three Phases of Memory Evolution	7
3	Agent 1 — Trigger Agent	8
3.1	Purpose	8
3.2	Data Sources	8
3.2.1	Google Trends (pytrends)	8
3.2.2	X / Twitter API v2 (Filtered Stream)	9
3.2.3	Reddit API (PRAW)	10
3.2.4	RSS / News Feed Crawler	11
3.3	Composite Opportunity Score	12
4	Agent 2 — Research Agent	13
4.1	Purpose	13
4.2	Five-Tier Source Hierarchy	14
4.3	Fact Verification Pipeline	15
5	Agent 3 — ICP Alignment Agent	17
5.1	Purpose	17
5.2	Three Scoring Criteria	17
5.3	The No-Brand-Push Rule	19
6	Agent 4 — SEO Agent	20
6.1	Purpose	20
6.2	Responsibilities	20
6.3	Breaking News Speed Mode	22
7	Agent 5 — Writer Agent	24
7.1	Purpose	24
7.2	Input Package	24
7.3	Hook Pattern Rules	24
7.4	Writer Agent Prompt Architecture	24
8	Human Review Gate	27

8.1	What It Is	27
8.2	What the Reviewer Sees	27
8.3	Reviewer Actions	27
9	Agent 6 — Repurposing Agent	28
9.1	Purpose	28
9.2	Platform-Specific Rules	28
9.2.1	LinkedIn Post	28
9.2.2	X Thread	28
9.2.3	Newsletter Article (LinkedIn “Web, AI & Automation Tips”)	28
10	Agent 7 — Scheduler and Publisher	31
10.1	Purpose	31
10.2	Optimal Publishing Schedule	31
10.3	Platform API Integration	31
11	Agent 8 — Analytics and Optimisation Agent	35
11.1	Purpose	35
11.2	Data Collection	35
11.3	2x2 Classification Matrix	36
11.4	Autonomous Optimisation Actions	36
11.5	Weekly Performance Digest	37
12	Complete Data Flow	38
12.1	End-to-End Flow: Mode B (Daily Post)	38
12.2	End-to-End Flow: Mode A (Breaking News)	38
12.3	Environment Variables Required	39
13	Dashboard Specification	40
13.1	What to Build	40
13.1.1	View 1: Topic Command Center	40
13.1.2	View 2: Human Review Panel	40
13.1.3	View 3: Analytics Dashboard	40
14	Key Business Rules	42
14.1	Rules That Must Never Be Violated	42
14.2	Content Quality Thresholds	42
15	Build Order and Dependencies	43
15.1	Recommended Build Sequence	43
15.2	External Services Checklist	43
16	Data Privacy and Compliance	44
16.1	What Data the System Stores	44
16.2	No PII Stored	44
16.3	API Token Security	44
A	Glossary	45
B	Current Baseline Metrics (May 2026)	46

C Top Performing Content Patterns

47

1. System Overview

1.1 What This System Does

Most tech companies with strong expertise struggle to convert that expertise into inbound leads. They post content that talks about themselves instead of solving client problems, guess what to write about instead of knowing what the market is searching for, and never find out what actually drove a lead.

This system fixes all three problems by running an autonomous pipeline that:

- **Detects** what topics are trending in real time (Google Trends, X, Reddit, RSS)
- **Filters** every topic through Bitcot's client persona — ensuring content speaks to CTOs, Founders, and VPs, not generic audiences
- **Researches** each topic using verified sources (Gartner, McKinsey, PubMed, etc.) and cross-checks all statistics
- **Writes** platform-native content in Bitcot's founder voice, never sounding like marketing
- **Publishes** via LinkedIn API, X API, and WordPress REST API at optimal times
- **Tracks** performance and feeds all learnings back into the system so every cycle gets smarter

1.2 Architecture at a Glance

The system has **8 agents**, **1 shared memory layer**, and **1 human review gate**.

#	Agent Name	Type	Runs
1	Trigger Agent	Fully automated	Always-on (hourly)
2	Research Agent	Fully automated	On trigger fire
3	ICP Alignment Agent	Fully automated	After research
4	SEO Agent	Fully automated	After ICP pass
5	Writer Agent	Fully automated	After SEO
–	Human Review Gate	Human in loop	Before repurposing
6	Repurposing Agent	Fully automated	After approval
7	Scheduler + Publisher	Fully automated	After repurposing
8	Analytics + Optimisation	Fully automated	Weekly

1.3 Two Operating Modes

Mode A — Breaking News (Automated)

Trigger: A topic spikes 3x its 7-day baseline volume in under 2 hours.

Flow: Trigger Agent → Research Agent → ICP Agent → Writer Agent → Human Review → Publish

Target time from spike to publish: Under 4 hours.

SEO Agent: Runs in speed mode (primary keyword only, no full cluster).

Mode B — Daily Scheduled Post

Trigger: Daily at 7:00 AM PST, OR manual topic selection from dashboard.

Flow: Full 8-agent pipeline runs in sequence.

Target time from signal to publish: 24–48 hours.

Includes: Full keyword cluster, internal linking map, newsletter format.

1.4 Tech Stack

Layer	Technology	Purpose
Orchestration	n8n <i>or</i> Python + LangGraph	Agent workflow automation
AI engine	Claude API (claude-sonnet-4-20250514)	All content generation and analysis
Signal collection SEO data	pytrends, X API v2, Reddit PRAW, feedparser DataForSEO API <i>or</i> Ahrefs API	Data ingestion Keyword research, SERP analysis
Publishing	LinkedIn API, X API v2, WordPress REST API	Content distribution
Memory layer	PostgreSQL + pgvector <i>or</i> Redis + JSON	Persistent state and learnings
Analytics	GA4 API, LinkedIn Analytics API, X Analytics API	Performance tracking
Dashboard	React + Next.js (frontend), FastAPI (backend)	Human review interface
Search Console	Google Search Console API	SEO monitoring
Scheduler	APScheduler (Python) <i>or</i> n8n cron nodes	Timed task execution

2. Memory Layer

2.1 What the Memory Layer Is

The memory layer is a **shared persistent context** that all 8 agents read from and write to. It is the system's brain. Without it, every agent starts from zero. With it, the system compounds — every publish cycle makes the next one smarter.

Think of it as a structured database that stores:

- Everything known about Bitcot (services, voice, ICP, competitors)
- Everything the system has published (topic, format, date, platform)
- Everything the analytics agent has learned (what drove leads, what flopped)

2.2 Data Schema

```
1 -- Core Bitcot context (seeded manually, rarely changes)
2 CREATE TABLE bitcot_context (
3   id SERIAL PRIMARY KEY,
4   service_list TEXT[],          -- ['AI agent dev', 'Healthcare
      software', ...]
5   icp_personas JSONB,          -- {title: 'CTO', pain: 'scaling',
      industry: 'healthcare'}
6   brand_voice_rules TEXT[],    -- ['Never open with brand name', '
      Use specific stats']
7   competitor_list TEXT[],     -- ['Intellectsoft', 'Rightpoint',
      ...]
8   approved_stats JSONB,       -- {claim: '...', source: '...', date
      : '...'}
9   updated_at TIMESTAMP
10 );
11
12 -- Content performance log (updated after every publish + analytics
    cycle)
13 CREATE TABLE content_log (
14   id SERIAL PRIMARY KEY,
15   topic TEXT NOT NULL,
16   hook_pattern TEXT,          -- 'stat_first' | 'scenario' | '
      reframe' | 'alarming_conditional'
17   platform TEXT,             -- 'linkedin' | 'x' | 'blog' | '
      newsletter'
18   format TEXT,               -- 'article' | 'image_post' | 'thread
      ' | 'carousel'
19   published_at TIMESTAMP,
20   impressions INTEGER,
21   engagement_rate DECIMAL(5,4),
22   clicks INTEGER,
23   ctr DECIMAL(5,4),
24   reposts INTEGER,
```

```

25     form_submissions INTEGER,      -- Conversion events attributed to
        this piece
26     icp_score DECIMAL(3,2),        -- 0.0 to 1.0      how well it matched
        ICP
27     vertical TEXT,                 -- 'healthcare_ai' | 'devops' | '
        ecommerce' | ...
28     converted_lead BOOLEAN DEFAULT FALSE
29 );
30
31 -- SEO state (updated weekly by analytics agent)
32 CREATE TABLE seo_state (
33     id SERIAL PRIMARY KEY,
34     url TEXT UNIQUE,
35     current_position DECIMAL(5,2),
36     impressions INTEGER,
37     clicks INTEGER,
38     ctr DECIMAL(5,4),
39     target_keyword TEXT,
40     last_optimised TIMESTAMP,
41     needs_refresh BOOLEAN DEFAULT FALSE,
42     updated_at TIMESTAMP
43 );
44
45 -- Topic scoring cache (updated daily by trigger agent)
46 CREATE TABLE topic_scores (
47     id SERIAL PRIMARY KEY,
48     topic TEXT NOT NULL,
49     google_trend_score INTEGER,      -- pytrends relative score 0-100
50     x_volume INTEGER,                -- tweet/post count in last 24h
51     reddit_upvotes INTEGER,
52     competitor_coverage BOOLEAN,    -- Has a competitor already published
        on this?
53     icp_relevance DECIMAL(3,2),      -- ICP agent pre-score
54     final_score DECIMAL(5,2),        -- composite score
55     created_at TIMESTAMP
56 );

```

Listing 2.1: Memory Layer Schema (PostgreSQL)

2.3 Three Phases of Memory Evolution

Phase 1 — Manual seed (now): Populate `bitcot_context` with services, personas, voice rules, competitors. System runs immediately on this baseline.

Phase 2 — After first analytics import: GA4 and LinkedIn data populates `content_log` with real performance. ICP scores recalibrate based on what actually drove enquiries, not assumptions.

Phase 3 — Compounding (3 months in): Memory self-updates every week. Writer agent, ICP agent, and SEO agent all read from a continuously improving dataset. Topics that drove leads get weighted higher. Failing formats get retired.

3. Agent 1 — Trigger Agent

3.1 Purpose

The Trigger Agent is the **always-on nervous system** of the entire pipeline. It runs 24/7, monitoring four real-time signal sources. Its only job is to decide: *is there something worth writing about right now, and how urgently?*

It fires in two modes:

- **Breaking news mode:** A keyword spikes 3x baseline in under 2 hours — fires instantly
- **Daily digest mode:** Every morning at 7:00 AM PST, delivers a ranked list of 6 topics to the dashboard

3.2 Data Sources

3.2.1 Google Trends (pytrends)

```
1 from pytrends.request import TrendReq
2 import pandas as pd
3
4 def get_trend_signals(keywords: list[str], geo: str = "US") -> dict:
5     """
6     Fetch rising and breakout keywords from Google Trends.
7     geo='US' keeps results US-specific (Bitcot's target market).
8     """
9     pytrends = TrendReq(hl='en-US', tz=360) # PST offset
10
11     # Build payload      max 5 keywords per request
12     pytrends.build_payload(
13         kw_list=keywords,
14         cat=0,
15         timeframe='now 7-d', # Last 7 days for baseline
16         geo=geo,
17         gprop='',
18     )
19
20     # Get interest over time
21     interest_df = pytrends.interest_over_time()
22
23     # Get related rising queries (these are the actual trending terms)
24     related = pytrends.related_queries()
25
26     results = {}
27     for kw in keywords:
28         rising = related.get(kw, {}).get('rising', pd.DataFrame())
29         results[kw] = {
30             'current_score': int(interest_df[kw].iloc[-1]) if not
31                 interest_df.empty else 0,
```

```

31         'week_avg': int(interest_df[kw].mean()) if not interest_df
32             .empty else 0,
33         'rising_queries': rising.to_dict('records') if not rising.
34             empty else []
35     }
36
37     return results
38
39 # Bitcot's core keyword groups to monitor
40 BITCOT_KEYWORDS = [
41     ['AI agent development', 'workflow automation', 'healthcare AI'],
42     ['custom software development', 'DevOps transformation', 'AI
    automation'],
43     ['telemedicine platform', 'FinTech software', 'mobile app
    development'],
44 ]

```

Listing 3.1: Google Trends signal collection

3.2.2 X / Twitter API v2 (Filtered Stream)

```

1  import tweepy
2  import time
3  from collections import deque
4  from datetime import datetime, timedelta
5
6  class XSpikeDetector:
7      """
8      Monitors X for volume spikes on tracked keywords.
9      Baseline is 7-day rolling average. Spike = 3x baseline in 2 hours.
10     """
11     def __init__(self, bearer_token: str):
12         self.client = tweepy.Client(bearer_token=bearer_token)
13         self.keyword_baselines = {} # {keyword: 7_day_avg_per_hour}
14         self.current_counts = {}    # {keyword: deque of hourly
15                                     counts}
16
17     def set_rules(self, keywords: list[str]):
18         """Set up filtered stream rules for Bitcot's topic areas."""
19         rules = []
20         for kw in keywords:
21             # Only track English tweets, exclude retweets for cleaner
22             # signal
23             rules.append(tweepy.StreamRule(
24                 f'"{kw}" lang:en -is:retweet has:links'
25             ))
26         self.client.add_rules(rules)
27
28     def calculate_spike(self, keyword: str, current_hour_count: int)
29     -> float:
30         """
31         Returns spike multiplier. 3.0 = 3x baseline = trigger Mode A.
32         """
33         baseline = self.keyword_baselines.get(keyword, 100)
34         if baseline == 0:
35             return 1.0
36         return current_hour_count / baseline

```

```

35     def check_spikes(self) -> list[dict]:
36         """Returns list of topics that have spiked above threshold."""
37         spikes = []
38         for kw, counts in self.current_counts.items():
39             if len(counts) >= 2:
40                 current = counts[-1]
41                 multiplier = self.calculate_spike(kw, current)
42                 if multiplier >= 3.0:
43                     spikes.append({
44                         'keyword': kw,
45                         'spike_multiplier': round(multiplier, 2),
46                         'mode': 'breaking_news',
47                         'urgency': 'immediate'
48                     })
49         return spikes

```

Listing 3.2: X API spike detection

3.2.3 Reddit API (PRAW)

```

1  import praw
2  from datetime import datetime, timezone
3
4  BITCOT_SUBREDDITS = [
5      'healthIT', 'MachineLearning', 'startups',
6      'cscareerquestions', 'devops', 'artificial',
7      'programming', 'SaaS', 'EntrepreneurRideAlong'
8  ]
9
10 def get_reddit_pain_points(reddit_client: praw.Reddit) -> list[dict]:
11     """
12     Fetches high-upvote posts from relevant subreddits.
13     These represent real buyer pain points in their own language.
14     High upvotes = validated community pain.
15     """
16     pain_points = []
17
18     for sub_name in BITCOT_SUBREDDITS:
19         sub = reddit_client.subreddit(sub_name)
20         # 'hot' gives currently trending, 'top' gives week's best
21         for post in sub.hot(limit=10):
22             if post.score > 50: # Minimum upvote threshold
23                 pain_points.append({
24                     'subreddit': sub_name,
25                     'title': post.title,
26                     'score': post.score,
27                     'num_comments': post.num_comments,
28                     'url': post.url,
29                     'selftext': post.selftext[:500], # First 500
30                                     chars
31                     'created_utc': datetime.fromtimestamp(
32                         post.created_utc, tz=timezone.utc
33                     ).isoformat()
34                 })
35
36     # Sort by score descending
37     return sorted(pain_points, key=lambda x: x['score'], reverse=True)

```

Listing 3.3: Reddit pain point collection

3.2.4 RSS / News Feed Crawler

```

1 import feedparser
2 from datetime import datetime, timezone
3 import time
4
5 INDUSTRY_FEEDS = [
6     'https://techcrunch.com/feed/',
7     'https://www.wired.com/feed/rss',
8     'https://feeds.feedburner.com/venturebeat/SZYF',
9     'https://www.healthcareitnews.com/rss.xml',
10    'https://www.fintechfutures.com/feed/',
11    'https://news.mit.edu/rss/topic/artificial-intelligence2',
12    'https://www.zdnet.com/news/rss.xml',
13 ]
14
15 BITCOT_KEYWORDS_LOWER = [
16     'ai agent', 'healthcare software', 'workflow automation',
17     'devops', 'telemedicine', 'fintech', 'mobile app',
18     'machine learning', 'generative ai', 'llm', 'digital health',
19 ]
20
21 def crawl_rss_feeds() -> list[dict]:
22     """
23     Fetches and filters RSS entries for Bitcot-relevant topics.
24     Returns only articles published in the last 24 hours.
25     """
26     relevant_articles = []
27     cutoff = time.time() - 86400 # 24 hours ago
28
29     for feed_url in INDUSTRY_FEEDS:
30         try:
31             feed = feedparser.parse(feed_url)
32             for entry in feed.entries:
33                 published = entry.get('published_parsed')
34                 if published and time.mktime(published) < cutoff:
35                     continue
36
37                 title_lower = entry.title.lower()
38                 summary_lower = entry.get('summary', '').lower()
39
40                 # Check if any Bitcot keyword appears
41                 matched_kw = [
42                     kw for kw in BITCOT_KEYWORDS_LOWER
43                     if kw in title_lower or kw in summary_lower
44                 ]
45
46                 if matched_kw:
47                     relevant_articles.append({
48                         'title': entry.title,
49                         'url': entry.link,
50                         'summary': entry.get('summary', '')[:400],
51                         'source': feed.feed.title,
52                         'published': entry.get('published', ''),
53                         'matched_keywords': matched_kw

```

```
54         })
55     except Exception as e:
56         print(f"Feed error {feed_url}: {e}")
57
58     return relevant_articles
```

Listing 3.4: RSS feed monitoring

3.3 Composite Opportunity Score

After collecting signals from all four sources, the Trigger Agent scores each topic:

$$\text{OpportunityScore} = (0.35 \times T_g) + (0.25 \times T_x) + (0.20 \times R_s) + (0.20 \times N_r) \quad (3.1)$$

Where:

- T_g = Google Trends score (0–100, normalised)
- T_x = X volume score (normalised to 0–100 against 30-day baseline)
- R_s = Reddit signal score (upvote count, normalised)
- N_r = News recency score (1.0 if under 6h, 0.7 if 6–24h, 0.3 if older)

Topics scoring above **70** are sent to Mode B (daily). Topics with a spike multiplier ≥ 3.0 fire Mode A immediately, bypassing scoring.

4. Agent 2 — Research Agent

4.1 Purpose

The Research Agent's job is to produce a **structured research brief** for every topic that passes from the Trigger Agent. This brief is the foundation for every other agent downstream. It must contain only verified, cited information — no hallucinated statistics ever reach the writer.

4.2 Five-Tier Source Hierarchy

Tier	Sources	Usage
1 — Primary	PubMed, arXiv, IEEE, WHO, CDC, World Bank, SEC filings	Gro- trut- Used as hard fact- Busi- bene- mar- Cite- with year Cur- nar- ra- tive tren- brea- ing con- text Gap anal- y- sis. Wha has been cov- ered vs. not cov- ered Buy lan- guag ver- ba- tims Rea pain poin
2 — Research firms	Gartner, McKinsey, Forrester, Statista, CB Insights, Deloitte	
3 — Industry news	TechCrunch, MIT Tech Review, VentureBeat, HealthIT News, Wired	
4 — Competitor content	Competitor blogs + LinkedIn posts	
5 — Community signals	G2 reviews, Reddit threads, Quora, LinkedIn comments	

4.3 Fact Verification Pipeline

Every claim collected must pass three checks before entering the brief:

1. **Cross-source check:** Statistic must appear in ≥ 2 independent sources. Single-source claims get tagged [SINGLE-SOURCE] — the writer can use them but must attribute carefully, not state as fact.
2. **Recency filter:** Data older than 18 months gets tagged [DATE: YYYY]. For breaking news content, only data under 72 hours is accepted as current.
3. **Confidence scoring:** Each claim gets one of three tags:
 - VERIFIED — appeared in 2+ independent Tier 1–2 sources
 - SINGLE-SOURCE — one source only, must be attributed
 - UNVERIFIED — cannot be confirmed, do not use in content

```

1 import anthropic
2 import json
3
4 client = anthropic.Anthropic()
5
6 def verify_claim(claim: str, sources: list[dict]) -> dict:
7     """
8     Uses Claude to cross-verify a claim against multiple sources.
9     Returns confidence tag and explanation.
10    """
11    source_text = "\n".join([
12        f"Source {i+1} ({s['tier']}) - {s['name']}: {s['excerpt']}"
13        for i, s in enumerate(sources)
14    ])
15
16    response = client.messages.create(
17        model="claude-sonnet-4-20250514",
18        max_tokens=500,
19        messages=[{
20            "role": "user",
21            "content": f"""You are a fact-checker. Evaluate this claim
22                        against the sources provided.
23
24            CLAIM: {claim}
25
26            SOURCES:
27            {source_text}
28
29            Respond ONLY with a JSON object:
30            {{
31                "confidence": "VERIFIED" | "SINGLE-SOURCE" | "UNVERIFIED",
32                "explanation": "brief reason",
33                "recommended_citation": "how to cite this claim correctly"
34            }}"""
35        })
36
37    return json.loads(response.content[0].text)
38
```



```

39 def build_research_brief(topic: str, all_claims: list[dict]) -> dict:
40     """
41     Assembles the final research brief that gets passed to ICP Agent.
42     Only VERIFIED and SINGLE-SOURCE claims are included.
43     """
44     verified = [c for c in all_claims if c['confidence'] != '
UNVERIFIED']
45
46     brief_response = client.messages.create(
47         model="claude-sonnet-4-20250514",
48         max_tokens=2000,
49         messages=[{
50             "role": "user",
51             "content": f"""Build a structured research brief for: {
topic}
52
53 VERIFIED CLAIMS:
54 {json.dumps(verified, indent=2)}
55
56 Return a JSON brief with these exact keys:
57 {{
58     "topic": str,
59     "core_argument": str,           // The strongest angle to write from
60     "target_audience": str,       // Who specifically should read this
61     "key_angles": [str],           // 3-5 different approaches to this
topic
62     "verified_stats": [{           // Stats ready to cite inline
63         "stat": str,
64         "source": str,
65         "year": int,
66         "confidence": str
67     }],
68     "competitor_gaps": [str],      // Topics competitors haven't covered
well
69     "buyer_language": [str],       // Real phrases from Reddit/G2 reviews
70     "counterarguments": [str],     // Opposing views (makes content richer
)
71     "image_brief": str,            // Visual description for designer/
image gen
72     "urgency_level": "breaking" | "high" | "normal"
73 }}"""
74     }]
75 )
76
77 return json.loads(brief_response.content[0].text)

```

Listing 4.1: Fact verification with Claude API

5. Agent 3 — ICP Alignment Agent

5.1 Purpose

The ICP (Ideal Client Profile) Alignment Agent is the **business relevance filter**. It sits between Research and SEO. Every topic must pass three criteria before proceeding. If a topic fails, the agent either **reshapes** it (rewrites the angle to be client-relevant) or **rejects** it.

The cardinal rule: Content must make the reader feel understood, not sold to. Never say Bitcot, show Bitcot. The ICP agent enforces this at every step.

5.2 Three Scoring Criteria

Criterion	Weight	What it checks
Persona match	35%	Does this topic speak to a CTO, VP Engineering, Founder, or Product Manager at a mid-sized US tech/healthcare/fintech company?
Pain point relevance	35%	Does the topic address a real pain the persona is experiencing right now? (architecture decisions, AI implementation, scaling, compliance, cost)
Service bridge	30%	Can Bitcot's expertise naturally and non-promotionally appear in this content? (AI agents, mobile apps, healthcare software, DevOps, workflow automation)

A topic must score ≥ 0.65 overall to proceed. Below that, the agent attempts a reshape. If reshape still scores below 0.50, the topic is rejected and the Trigger Agent is asked for the next ranked topic.

```
1 import anthropic
2 import json
3
4 client = anthropic.Anthropic()
5
6 BITCOT_ICP = {
7     "target_personas": [
8         {"title": "CTO", "needs": ["scalability", "AI integration", "ROI", "architecture"]},
9         {"title": "VP Engineering", "needs": ["delivery speed", "tech debt", "team scaling"]},
10        {"title": "Founder/CEO", "needs": ["speed to market", "cost efficiency", "trust"]},
11        {"title": "Product Manager", "needs": ["build vs buy", "timelines", "user retention"]},
```

```

12 ],
13 "target_industries": ["healthcare", "fintech", "enterprise SaaS",
14 "startups", "ecommerce"],
15 "target_geography": "United States (primary: California, San Diego)",
16 "bitcot_services": [
17     "AI agent development", "web and mobile app development",
18     "workflow automation", "GenAI integration", "healthcare software",
19     "FinTech solutions", "DevOps and cloud", "custom software development"
20 ],
21 "brand_rules": [
22     "Never open with Bitcot's name",
23     "Maximum 1 brand mention per piece, only in author bio or earned closing line",
24     "Content must be insight-led, never promotional",
25     "Reader should feel the writer understands their world",
26     "No CTAs that say 'Contact Bitcot today' or 'Get a free quote'"
27 ]
28 }
29
30 def score_topic(research_brief: dict) -> dict:
31     """
32     Scores a research brief against Bitcot's ICP.
33     Returns score, decision, and reshaped angle if needed.
34     """
35     response = client.messages.create(
36         model="claude-sonnet-4-20250514",
37         max_tokens=1500,
38         messages=[{
39             "role": "user",
40             "content": f"""You are the ICP Alignment Agent for Bitcot Technology.
41
42 BITCOT ICP DEFINITION:
43 {json.dumps(BITCOT_ICP, indent=2)}
44
45 RESEARCH BRIEF TO EVALUATE:
46 Topic: {research_brief['topic']}
47 Core argument: {research_brief['core_argument']}
48 Target audience: {research_brief['target_audience']}
49
50 Score this topic on three criteria (0.0 to 1.0 each):
51 1. persona_match: Does it speak to Bitcot's ideal buyer?
52 2. pain_relevance: Does it address a real, current pain point?
53 3. service_bridge: Can Bitcot's expertise appear naturally (not promotionally)?
54
55 Then calculate: overall_score = (persona_match * 0.35) + (
56     pain_relevance * 0.35) + (service_bridge * 0.30)
57
58 Decision rules:
59 - overall >= 0.65: PASS
60 - 0.50 <= overall < 0.65: RESHAPE (provide a new, better angle)
61 - overall < 0.50: REJECT

```

```

61 Respond ONLY with JSON:
62 {{
63     "persona_match": float,
64     "pain_relevance": float,
65     "service_bridge": float,
66     "overall_score": float,
67     "decision": "PASS" | "RESHAPE" | "REJECT",
68     "persona_tag": "CTO" | "Founder" | "VP Engineering" | "PM",
69     "reshaped_angle": str or null, // Only if RESHAPE
70     "rejection_reason": str or null, // Only if REJECT
71     "content_format_recommendation": "article" | "image_post" | "thread",
72     "brand_push_risk": "low" | "medium" | "high"
73 }} ""
74     }}
75 )
76
77 return json.loads(response.content[0].text)

```

Listing 5.1: ICP Alignment Agent scoring

5.3 The No-Brand-Push Rule

This rule is enforced by the ICP Agent and re-checked by the Writer Agent.

NEVER allowed:

- Opening or closing with “Bitcot” or “At Bitcot, we...”
- “Contact Bitcot today”, “Get a free quote”, “Our team of 200+ engineers”
- Any sentence that reads like an advertisement
- More than one brand mention per content piece

ALLOWED (organic, earned):

- “We’ve shipped 40+ agents and seen this pattern every time” (founder voice)
- “In our experience working with healthcare clients in California...”
- Author bio at bottom of blog only
- Final tweet in an X thread: “— Raj @ Bitcot” (signature only)

The goal: The reader should finish the content thinking “this person understands my world” — not “this company is trying to sell me something.”

6. Agent 4 — SEO Agent

6.1 Purpose

The SEO Agent is the **master of search strategy**. It receives the ICP-approved brief and produces a complete SEO package that the Writer Agent uses to structure the content. It does not write content — it tells the writer exactly how to structure it for maximum search performance.

6.2 Responsibilities

- Find the **primary keyword** — highest intent, achievable difficulty for Bitcot's domain authority
- Build a **keyword cluster** of 6–10 semantically related terms
- Determine **search intent**: informational (blog) / commercial (landing page) / navigational (FAQ)
- Run **SERP gap analysis**: find where competitors rank with thin or outdated content
- Map **internal linking opportunities**: which existing pages this new content should link to and receive links from
- Set **title tag**, **meta description**, **H1**, **H2 structure**, recommended word count

```
1 import requests
2 import json
3 import base64
4
5 class SEOAgent:
6     def __init__(self, login: str, password: str):
7         self.auth = base64.b64encode(
8             f"{login}:{password}".encode()
9         ).decode()
10        self.base_url = "https://api.dataforseo.com/v3"
11
12    def get_keyword_data(self, keyword: str, location_code: int =
13        2840) -> dict:
14        """
15        location_code 2840 = United States
16        Gets search volume, competition, CPC for a keyword.
17        """
18        payload = [{
19            "keywords": [keyword],
20            "location_code": location_code,
21            "language_code": "en"
22        }]
23
24        response = requests.post(
```

```

24         f"{self.base_url}/keywords_data/google_ads/search_volume/
           live",
25         headers={
26             "Authorization": f"Basic {self.auth}",
27             "Content-Type": "application/json"
28         },
29         data=json.dumps(payload)
30     )
31     return response.json()
32
33 def get_serp_analysis(self, keyword: str) -> dict:
34     """
35     Fetches current SERP for a keyword.
36     Returns top 10 results with their estimated traffic and
37     content quality.
38     Used to find gap opportunities.
39     """
40     payload = [{
41         "keyword": keyword,
42         "location_code": 2840,
43         "language_code": "en",
44         "calculate_rectangles": True
45     }]
46
47     response = requests.post(
48         f"{self.base_url}/serp/google/organic/live/advanced",
49         headers={
50             "Authorization": f"Basic {self.auth}",
51             "Content-Type": "application/json"
52         },
53         data=json.dumps(payload)
54     )
55     return response.json()
56
57 def build_seo_package(
58     self,
59     research_brief: dict,
60     icp_result: dict,
61     existing_blog_urls: list[str]
62 ) -> dict:
63     """
64     Main SEO Agent function. Returns complete SEO package for
65     writer.
66     In breaking news mode: returns minimal package (primary KW
67     only).
68     """
69     topic = research_brief['topic']
70     persona = icp_result['persona_tag']
71
72     # 1. Find primary keyword
73     kw_data = self.get_keyword_data(topic)
74
75     # 2. Get SERP landscape
76     serp = self.get_serp_analysis(topic)
77
78     # 3. Build internal link map from existing URLs
79     # Simple heuristic: find URLs containing related terms
80     related_pages = [

```

```

78         url for url in existing_blog_urls
79         if any(word in url for word in topic.lower().split())
80     ]
81
82     # 4. Use Claude to synthesise the full SEO package
83     claude_client = anthropic.Anthropic()
84
85     response = claude_client.messages.create(
86         model="claude-sonnet-4-20250514",
87         max_tokens=1500,
88         messages=[{
89             "role": "user",
90             "content": f"""You are the SEO Agent for Bitcot
91                         Technology.
92
93 TOPIC: {topic}
94 PERSONA: {persona}
95 KEYWORD DATA: {json.dumps(kw_data, indent=2)[:1000]}
96 SERP LANDSCAPE: {json.dumps(serp, indent=2)[:1000]}
97 RELATED EXISTING PAGES: {related_pages[:10]}
98
99 Build a complete SEO package. Return ONLY JSON:
100 {{
101     "primary_keyword": str,
102     "keyword_cluster": [str],          // 6-10 related terms to include
103                             naturally
104     "search_intent": "informational" | "commercial" | "navigational",
105     "content_format": "blog_post" | "landing_page" | "faq",
106     "recommended_word_count": int,
107     "title_tag": str,                  // Under 60 characters
108     "meta_description": str,           // Under 155 characters
109     "h1": str,
110     "h2_structure": [str],             // Ordered list of H2 headings
111     "internal_links_to_add": [str],    // Existing URLs to link FROM this
112                                         page
113     "internal_links_to_receive": [str], // Existing URLs that should
114                                         link TO this page
115     "serp_gap": str,                   // What competitors are doing weakly
116     "speed_mode": false                 // Set true only for breaking news
117 }}"""
118
119         })
120
121     return json.loads(response.content[0].text)

```

Listing 6.1: SEO Agent using DataForSEO API

6.3 Breaking News Speed Mode

In Mode A (breaking news), the SEO Agent returns a reduced package:

- **Primary keyword only** — no cluster build
- **Basic H2 structure** — 3–4 headings max
- **No internal linking** — added in post-publish update

- **Target publish time:** Under 4 hours from spike detection
- **Post-publish:** Full SEO optimisation runs within 48 hours

7. Agent 5 — Writer Agent

7.1 Purpose

The Writer Agent is the **ghostwriter**. It receives the full package from Research + ICP + SEO and produces the primary content piece. It writes in Raj Sanghvi's (Bitcot founder) voice — insight-led, authoritative, never promotional.

7.2 Input Package

The Writer Agent receives:

1. Research brief (verified stats, buyer language, key angles)
2. ICP result (persona tag, brand rules, format recommendation)
3. SEO package (keyword cluster, H2 structure, word count, title tag)
4. Memory layer context (voice rules, past winning hook patterns)

7.3 Hook Pattern Rules

Based on analysis of 36 Bitcot posts, the following hook patterns have been empirically validated:

Pattern	Structure	Avg Engagement	Example
A — Stat first	Specific number in line 1	3.2%	“70% of healthcare AI in
B — Reader scenario	Their situation, line 1	3.5% CTR	“Why are teams still usin
C — Contrarian reframe	[X] isn't [Y], it's [Z]	2.8%	“Your team isn't slow. Y
D — Alarming conditional	What if [bad thing]?	3.3%	“Is your pipeline shippin

NEVER use these failing hook patterns:

- Vague aspiration: “Still relying on good enough software?” (0.54% engagement — lowest of 36 posts)
- Product announcement: “With the latest innovations in [Tool], businesses can now...” (1.04%)
- Generic cost warning: “Are you unknowingly overspending on [X]?” (0.87%)

These patterns describe every company equally and resonate with none.

7.4 Writer Agent Prompt Architecture

```
1 def generate_blog_post(  
2     research_brief: dict,  
3     icp_result: dict,
```

```

4     seo_package: dict,
5     memory_context: dict,
6     personal_input: dict = None # Only in manual input mode
7 ) -> dict:
8     """
9     Generates blog post. In personal input mode, uses the founder's
10    own angle/opinion and wraps the research around it.
11    """
12
13    # Determine hook pattern from memory
14    best_hooks = memory_context.get('winning_hook_patterns', ['
15        stat_first', 'scenario'])
16
17    # Build context string for prompt
18    verified_stats = "\n".join([
19        f"- {s['stat']} (Source: {s['source']}, {s['year']}, {s['
20        confidence']})"
21        for s in research_brief['verified_stats']
22    ])
23
24    personal_section = ""
25    if personal_input:
26        personal_section = f"""
27    FOUNDER'S PERSONAL INPUT:
28    Opinion/angle: {personal_input.get('opinion', '')}
29    Key point to make: {personal_input.get('key_point', '')}
30    Image idea (describe it): {personal_input.get('image_idea', '')}
31    Tone preference: {personal_input.get('tone', 'thought_leader')}
32
33    Build the blog around this personal perspective. The research should
34    support the founder's view, not replace it.
35    """
36
37    response = client.messages.create(
38        model="claude-sonnet-4-20250514",
39        max_tokens=4000,
40        system=f"""You are ghostwriting for Raj Sanghvi, founder of
41        Bitcot Technology, San Diego.
42
43    VOICE RULES:
44    {json.dumps(memory_context.get('brand_voice_rules', []), indent=2)}
45
46    BRAND RULES (CRITICAL):
47    - Maximum ONE brand mention in the entire post
48    - It must appear only in the closing line or author bio
49    - NEVER open with 'At Bitcot...' or 'Bitcot helps...'
50    - The post must read as expert insight, not company marketing
51    - Use 'we' only to describe real client experiences, not to promote
52      services
53
54    WINNING HOOK PATTERNS TO USE:
55    {json.dumps(best_hooks, indent=2)}""",
56
57    messages=[{
58        "role": "user",
59        "content": f"""Write a complete blog post.
60
61    TOPIC: {research_brief['topic']}

```

```

58 TARGET_PERSONA: {icp_result['persona_tag']}
59 TARGET_WORD_COUNT: {seo_package['recommended_word_count']}
60
61 SEO_REQUIREMENTS:
62 - Primary keyword: {seo_package['primary_keyword']}
63 - Include naturally: {'', '.join(seo_package['keyword_cluster'])}
64 - H1: {seo_package['h1']}
65 - H2 structure to follow: {json.dumps(seo_package['h2_structure'])}
66
67 VERIFIED_STATS_TO_USE (cite these exactly):
68 {verified_stats}
69
70 BUYER_LANGUAGE_TO_WEAVE_IN (use these exact phrases):
71 {json.dumps(research_brief.get('buyer_language', []))}
72
73 CORE_ARGUMENT:
74 {research_brief['core_argument']}
75 {personal_section}
76
77 FORMAT_REQUIREMENTS:
78 - Open with hook pattern A, B, C, or D (never a vague statement)
79 - Use short paragraphs (2-3 sentences max)
80 - Each H2 section ends with a practical, actionable tip
81 - Final section: one soft CTA (NOT 'Contact us', NOT 'Get a free quote
82   ')
83   Example: 'If you're navigating this, I'm happy to share what we've
84     seen --- drop a comment.'
85 - Flag any uncertain claims with [NEEDS HUMAN CHECK]
86
87 Also output:
88 - image_brief: Describe the ideal image for this post (style, text
89   overlay, mood, colours, Bitcot branding placement)
90 - meta_title: Under 60 chars
91 - meta_description: Under 155 chars"""
92   ]]
93 )
94
95 return {
96     'blog_content': response.content[0].text,
97     'seo_package': seo_package,
98     'icp_result': icp_result,
99     'research_brief': research_brief,
100     'status': 'pending_human_review'
101 }

```

Listing 7.1: Writer Agent with full context injection

8. Human Review Gate

8.1 What It Is

The Human Review Gate is the **only point where a human must act** before content is published. It is not an agent — it is a dashboard interface. Everything before and after is autonomous.

8.2 What the Reviewer Sees

The dashboard presents, side-by-side:

1. Blog post draft (full text, editable inline)
2. LinkedIn post preview (generated by Repurposing Agent preview)
3. X thread preview (generated by Repurposing Agent preview)
4. Image brief (text description of the visual to create)
5. ICP score and persona tag
6. Flagged claims tagged [NEEDS HUMAN CHECK]

8.3 Reviewer Actions

Action	Keyboard shortcut	What happens
Approve	Ctrl+Enter	Passes to Repurposing Agent
Edit inline	Click any paragraph	Direct text editing, auto-saves
Regenerate section	Ctrl+R on section	Rewrites that H2 section only
Change tone	Dropdown menu	Regenerates in selected tone
Reject topic	Ctrl+D	Flags topic, asks Trigger Agent for next

This gate must never be removed or auto-bypassed. Even in breaking news mode (Mode A), a human reviews before publication. The time saved by automation makes the human review fast — not eliminated.

9. Agent 6 — Repurposing Agent

9.1 Purpose

The Repurposing Agent takes the approved blog post and converts it into **platform-native formats**. Critically, each format is not a copy-paste or truncation of the blog — it is a full rewrite engineered for that platform’s specific algorithm and audience behaviour.

9.2 Platform-Specific Rules

9.2.1 LinkedIn Post

- **Length:** 200–300 words
- **Structure:** Hook line (must be Pattern A, B, C, or D) → 3–4 insight paragraphs (2 sentences each) → one soft question or CTA
- **Hashtags:** Maximum 3. Must be highly relevant, not keyword-stuffed
- **Brand:** One mention maximum. Signature: “— Raj | Bitcot” at end only
- **Algorithm notes:** LinkedIn rewards dwell time (long posts read slowly) and comments (ask a question at the end). Avoid links in the post body — they suppress reach. Put the blog link in the first comment instead

9.2.2 X Thread

- **Length:** 6–8 tweets
- **Structure:** Tweet 1 = hook (must stand alone and be irresistible) → Tweets 2–6 = one insight per tweet, numbered, self-contained → Tweet 7 = CTA with DM hook (e.g., “DM me AGENT COST and I’ll send our breakdown”)
- **Tone:** More punchy and opinionated than LinkedIn. Short sentences. Contrarian edge where appropriate
- **DM hook:** This is a lead capture mechanic. The keyword response (e.g., “AGENT COST”) triggers a direct message with a resource. This is a proven high-conversion tactic on X
- **Algorithm notes:** X rewards replies and bookmarks. The thread structure (numbered tweets) encourages people to save and return. Hook tweet drives initial reach; thread depth drives saves

9.2.3 Newsletter Article (LinkedIn “Web, AI & Automation Tips”)

- **Audience:** 5,871 subscribers — warm, opted-in, high-value
- **Length:** 600–900 words (longer than a LinkedIn post, shorter than full blog)

- **Structure:** Direct insight opening → 3 key points with subheadings → Actionable takeaway → Link to full blog post
- **Tone:** More personal than blog, more detailed than LinkedIn post. This audience chose to subscribe — speak to them directly

```

1 import asyncio
2 import anthropic
3
4 async def generate_linkedin_post(blog_content: str, research_brief:
    dict,
5                                     memory_context: dict) -> str:
6     client = anthropic.Anthropic()
7     response = client.messages.create(
8         model="claude-sonnet-4-20250514",
9         max_tokens=800,
10        system="""You are writing a LinkedIn post. Platform rules:
11- 200-300 words maximum
12- Open with Pattern A (stat), B (scenario), C (reframe), or D (
    alarming conditional)
13- 3-4 paragraphs of 2 sentences each
14- End with ONE soft question or CTA (not a sales pitch)
15- Maximum 3 hashtags at the end
16- Maximum 1 brand mention (signature only at the very end)
17- DO NOT include a URL in the post body (put in first comment instead)
18- LinkedIn rewards dwell time: write for someone to read slowly""",
19        messages=[{
20            "role": "user",
21            "content": f"""Convert this blog into a LinkedIn post.
22
23BLOG CONTENT:
24{blog_content[:3000]}
25
26WINNING HOOK PATTERNS:
27{memory_context.get('winning_hook_patterns', [])}
28
29KEY STATS TO INCLUDE:
30{[s['stat'] for s in research_brief.get('verified_stats', [])[:3]]}""",
31        })
32    )
33    return response.content[0].text
34
35 async def generate_x_thread(blog_content: str, research_brief: dict)
    -> list[str]:
36     client = anthropic.Anthropic()
37     response = client.messages.create(
38         model="claude-sonnet-4-20250514",
39         max_tokens=1200,
40         system="""You are writing an X (Twitter) thread. Platform
        rules:
41- 6-8 tweets
42- Tweet 1: Hook only. Must be irresistible as a standalone. No context
    needed.
43- Tweets 2-6: Numbered. One insight per tweet. Self-contained. Short
    sentences.
44- Tweet 7: CTA with DM hook keyword in ALL CAPS (e.g., 'DM me AGENT
    COST')
45- Tone: punchy, opinionated, contrarian where appropriate

```

```

46 - X rewards replies and bookmarks: make each tweet quotable
47 - Return as a JSON array of tweet strings"",
48     messages=[{
49         "role": "user",
50         "content": f"Convert this blog to an X thread:\n\n{
                    blog_content[:3000]}"
51     }]
52 )
53 import json
54 try:
55     return json.loads(response.content[0].text)
56 except:
57     # Fallback: split by tweet markers
58     return response.content[0].text.split('\n\n')
59
60 async def repurpose_all_platforms(
61     approved_blog: dict,
62     research_brief: dict,
63     memory_context: dict
64 ) -> dict:
65     """
66     Generates all platform formats in parallel.
67     All run simultaneously to save time.
68     """
69     blog_content = approved_blog['blog_content']
70
71     linkedin_task = generate_linkedin_post(blog_content,
72     research_brief, memory_context)
73     x_thread_task = generate_x_thread(blog_content, research_brief)
74
75     linkedin_post, x_thread = await asyncio.gather(linkedin_task,
76     x_thread_task)
77
78     return {
79         'linkedin': linkedin_post,
80         'x_thread': x_thread,
81         'blog': blog_content,
82         'image_brief': approved_blog.get('image_brief', ''),
83         'status': 'ready_to_schedule'
84     }

```

Listing 9.1: Repurposing Agent — parallel platform generation

10. Agent 7 — Scheduler and Publisher

10.1 Purpose

The Scheduler and Publisher handles **all API integrations and publish timing**. It staggers posts across platforms to avoid audience fatigue and maximise each platform's reach window.

10.2 Optimal Publishing Schedule

Platform	Optimal time (PST)	Mode A (breaking)	Notes
LinkedIn	9:00 AM Tue–Thu	Within 30 min of approval	Publish first — professional audience
X thread	11:00 AM same day	30 min after LinkedIn	Stagger to avoid cannibalization
Blog	10:00 AM next day	Within 4 hours	Needs SEO indexing time
Newsletter	8:00 AM Thursday	N/A (scheduled only)	Newsletter has its own weekly cadence

10.3 Platform API Integration

```
1 import requests
2
3 def publish_linkedin_post(
4     access_token: str,
5     organization_urn: str, # e.g., 'urn:li:organization:12345678'
6     post_text: str,
7     first_comment_url: str = None
8 ) -> dict:
9     """
10     Publishes a text post to Bitcot's LinkedIn company page.
11     Blog URL goes in first comment, not post body (to avoid reach
12     suppression).
13     """
14     headers = {
15         'Authorization': f'Bearer {access_token}',
16         'Content-Type': 'application/json',
17         'X-Restli-Protocol-Version': '2.0.0'
18     }
19
20     payload = {
21         "author": organization_urn,
22         "lifecycleState": "PUBLISHED",
23         "specificContent": {
24             "com.linkedin.ugc.ShareContent": {
25                 "shareCommentary": {
26                     "text": post_text
```



```

27         "shareMediaCategory": "NONE"
28     }
29 },
30 "visibility": {
31     "com.linkedin.ugc.MemberNetworkVisibility": "PUBLIC"
32 }
33 }
34
35 response = requests.post(
36     'https://api.linkedin.com/v2/ugcPosts',
37     headers=headers,
38     json=payload
39 )
40
41 result = response.json()
42 post_id = result.get('id', '')
43
44 # If URL provided, add as first comment immediately
45 if first_comment_url and post_id:
46     add_linkedin_comment(access_token, post_id,
47                          f"Full breakdown here: {first_comment_url}"
48                          )
49
50 return result
51
52 def publish_x_thread(
53     bearer_token: str,
54     tweets: list[str]
55 ) -> list[str]:
56     """
57     Publishes a thread on X. Each tweet replies to the previous.
58     Returns list of tweet IDs.
59     """
60     import tweepy
61     client = tweepy.Client(bearer_token=bearer_token)
62
63     tweet_ids = []
64     reply_to_id = None
65
66     for i, tweet_text in enumerate(tweets):
67         if i == 0:
68             response = client.create_tweet(text=tweet_text)
69         else:
70             response = client.create_tweet(
71                 text=tweet_text,
72                 in_reply_to_tweet_id=reply_to_id
73             )
74
75         tweet_ids.append(response.data['id'])
76         reply_to_id = response.data['id']
77
78     return tweet_ids
79
80 def publish_wordpress_post(
81     wp_url: str,
82     username: str,
83     app_password: str,
84     title: str,

```

```

84     content: str,
85     meta_description: str,
86     categories: list[int] = None
87 ) -> dict:
88     """
89     Publishes blog post to WordPress via REST API.
90     Also submits to Google Search Console for instant indexing.
91     """
92     import base64
93
94     credentials = base64.b64encode(
95         f"{username}:{app_password}".encode()
96     ).decode()
97
98     headers = {
99         'Authorization': f'Basic {credentials}',
100         'Content-Type': 'application/json'
101     }
102
103     payload = {
104         'title': title,
105         'content': content,
106         'status': 'publish',
107         'categories': categories or [],
108         'meta': {
109             'description': meta_description
110         }
111     }
112
113     response = requests.post(
114         f"{wp_url}/wp-json/wp/v2/posts",
115         headers=headers,
116         json=payload
117     )
118
119     post_data = response.json()
120
121     # Submit URL to Google Search Console for instant indexing
122     if post_data.get('link'):
123         submit_to_search_console(post_data['link'])
124
125     return post_data
126
127 def anti_cannibalization_check(
128     proposed_topic: str,
129     recent_posts: list[dict],
130     window_days: int = 5
131 ) -> bool:
132     """
133     Returns True if it's safe to publish.
134     Prevents posting similar topics within the window.
135     """
136     from datetime import datetime, timedelta
137     cutoff = datetime.now() - timedelta(days=window_days)
138
139     recent = [p for p in recent_posts if p['published_at'] > cutoff]
140
141     # Use Claude to check semantic similarity

```

```
142     if not recent:
143         return True
144
145     response = client.messages.create(
146         model="claude-sonnet-4-20250514",
147         max_tokens=100,
148         messages=[{
149             "role": "user",
150             "content": f"Is '{proposed_topic}' semantically similar
151                          (same core subject)
152                          to any of these recent topics: {[p['topic'] for p in recent]}?
153                          Respond with only: YES or NO"
154         }]
155     )
156     return response.content[0].text.strip() == "NO"
```

Listing 10.1: LinkedIn API publishing

11. Agent 8 — Analytics and Optimisation Agent

11.1 Purpose

The Analytics and Optimisation Agent is the **learning engine**. It runs weekly, pulls performance data from all platforms, classifies every content piece into the 2x2 matrix, identifies underperformers, and writes updates back to the memory layer.

11.2 Data Collection

```
1 from google.analytics.data_v1beta import BetaAnalyticsDataClient
2 from google.analytics.data_v1beta.types import (
3     RunReportRequest, DateRange, Metric, Dimension
4 )
5
6 def pull_ga4_data(property_id: str, start_date: str, end_date: str) ->
7     list[dict]:
8     """
9     Pulls blog performance data from GA4.
10    property_id format: '123456789'
11    """
12    client = BetaAnalyticsDataClient()
13
14    request = RunReportRequest(
15        property=f"properties/{property_id}",
16        date_ranges=[DateRange(start_date=start_date, end_date=
17            end_date)],
18        metrics=[
19            Metric(name="sessions"),
20            Metric(name="bounceRate"),
21            Metric(name="averageSessionDuration"),
22            Metric(name="conversions"), # Form submissions
23        ],
24        dimensions=[
25            Dimension(name="pagePath"),
26            Dimension(name="pageTitle"),
27        ]
28    )
29
30    response = client.run_report(request)
31
32    results = []
33    for row in response.rows:
34        results.append({
35            'url': row.dimension_values[0].value,
36            'title': row.dimension_values[1].value,
37            'sessions': int(row.metric_values[0].value),
38            'bounce_rate': float(row.metric_values[1].value),
39            'avg_duration': float(row.metric_values[2].value),
```

```

38         'conversions': int(row.metric_values[3].value),
39     })
40
41     return results

```

Listing 11.1: Analytics Agent — multi-platform data pull

11.3 2x2 Classification Matrix

Every content piece is classified weekly:

		High Conversion	Low Conversion
High	Engage- ment	Q1: Double down. Replicate this topic, format, and hook pattern in future content. Feed to memory as winning template.	Q3: Fix the CTA. Content resonates but doesn't move people to action. Rewrite the closing CTA. Add a more specific offer.
Low	Engage- ment	Q2: Fix distribution. Content converts when people find it but reach is low. Boost with internal links, cross-posting, or paid amplification.	Q4: Rethink entirely. Both engagement and conversion are weak. The topic, hook, or format is wrong. Retire or completely rewrite.

11.4 Autonomous Optimisation Actions

```

1  def optimise_underperforming_page(
2      page_data: dict,
3      seo_state: dict,
4      reason: str
5  ) -> dict:
6      """
7      Automatically rewrites metadata and optionally refreshes content
8      for underperforming pages. No human review required for metadata.
9      Content rewrites are flagged for human review before publishing.
10     """
11     response = client.messages.create(
12         model="claude-sonnet-4-20250514",
13         max_tokens=800,
14         messages=[{
15             "role": "user",
16             "content": f"""Optimise this underperforming page.
17
18     PAGE: {page_data['url']}
19     CURRENT TITLE: {page_data['title']}
20     CURRENT CTR: {seo_state['ctr']}%
21     CURRENT POSITION: {seo_state['current_position']}
22     REASON FOR OPTIMISATION: {reason}
23     IMPRESSIONS: {seo_state['impressions']} (page is seen but not clicked)
24
25     Provide:
26     1. New title tag (under 60 chars, stronger hook, includes primary
27         keyword)
28     2. New meta description (under 155 chars, includes clear value prop)

```

```

28 3. Assessment: does the full content need refreshing? (yes/no + reason
   )
29 4. If yes: what specific sections need updating?
30
31 Return as JSON. """
32     }]
33 )
34
35 import json
36 return json.loads(response.content[0].text)
37
38 def update_memory_from_analytics(
39     content_log_entries: list[dict],
40     db_connection
41 ) -> None:
42     """
43     After each analytics cycle, updates the memory layer.
44     This is how the system gets smarter over time.
45     """
46     for entry in content_log_entries:
47         # Update topic weights based on real performance
48         if entry['conversions'] > 0:
49             # This topic drove a real lead --- increase its weight
50             db_connection.execute("""
51                 UPDATE topic_scores
52                 SET final_score = final_score * 1.2
53                 WHERE topic ILIKE %s
54             """, [f"%{entry['vertical']}%"])
55
56         if entry['engagement_rate'] < 0.005: # Below 0.5%
57             # This hook pattern failed --- flag it
58             db_connection.execute("""
59                 UPDATE content_log
60                 SET hook_pattern_outcome = 'failed'
61                 WHERE id = %s
62             """, [entry['id']])

```

Listing 11.2: Optimisation Agent — automated content refresh

11.5 Weekly Performance Digest

Every Monday at 8:00 AM PST, the Analytics Agent sends a digest to the dashboard with:

- Top 3 performing pieces this week (by engagement and by conversion)
- Bottom 3 performers (flagged for review or retirement)
- Trending topics the system detected but did not act on (missed opportunities)
- Memory layer update summary (what changed in the scoring weights)
- SEO movement report (which pages moved up or down in Google rankings)

12. Complete Data Flow

12.1 End-to-End Flow: Mode B (Daily Post)

1. **7:00 AM PST** — Trigger Agent runs all 4 signal sources
2. Composite opportunity scores calculated, top 6 topics ranked
3. Topics sent to dashboard as “Today’s Topic Cards”
4. **Human selects** a topic (or enters personal input mode with their own angle)
5. Research Agent fires: pulls from 5-tier source hierarchy, verifies all claims, builds brief
6. ICP Agent scores the brief: PASS / RESHAPE / REJECT
7. If PASS: SEO Agent runs keyword research, SERP gap analysis, builds H2 structure
8. Writer Agent generates full blog post + image brief + meta data
9. **Human Review Gate:** reviewer approves, edits, or rejects
10. If approved: Repurposing Agent generates LinkedIn, X thread, newsletter (parallel)
11. Scheduler queues: LinkedIn at 9:00 AM, X at 11:00 AM, Blog next morning
12. Publisher sends all posts via respective APIs
13. Content logged to memory layer with all metadata
14. Analytics Agent picks it up in the next weekly cycle

12.2 End-to-End Flow: Mode A (Breaking News)

1. Trigger Agent detects 3x spike on a keyword
2. Push notification sent to reviewer immediately
3. Research Agent fires in speed mode (45 min max)
4. ICP Agent: quick score only, no reshape delay
5. Writer Agent: standard blog + lightweight SEO (no full cluster)
6. **Human Review Gate:** reviewer must act within 30 minutes
7. Repurposing Agent: LinkedIn + X only (blog can follow within 4 hours)
8. Publish LinkedIn and X immediately on approval
9. Blog published within 4 hours

10. Full SEO optimisation run on blog within 48 hours

12.3 Environment Variables Required

```
1 # Anthropic
2 ANTHROPIC_API_KEY=sk-ant-...
3
4 # Google Trends (no key needed, uses pytrends)
5 # X / Twitter
6 X_BEARER_TOKEN=AAAA...
7 X_API_KEY=...
8 X_API_SECRET=...
9 X_ACCESS_TOKEN=...
10 X_ACCESS_TOKEN_SECRET=...
11
12 # Reddit
13 REDDIT_CLIENT_ID=...
14 REDDIT_CLIENT_SECRET=...
15 REDDIT_USER_AGENT=BitcotContentBot/1.0
16
17 # SEO
18 DATAFORSEO_LOGIN=...
19 DATAFORSEO_PASSWORD=...
20
21 # LinkedIn
22 LINKEDIN_ACCESS_TOKEN=...
23 LINKEDIN_ORGANIZATION_URN=urn:li:organization:...
24
25 # WordPress
26 WP_URL=https://bitcot.com
27 WP_USERNAME=...
28 WP_APP_PASSWORD=...
29
30 # Google Analytics
31 GA4_PROPERTY_ID=...
32 GOOGLE_APPLICATION_CREDENTIALS=/path/to/service-account.json
33
34 # Google Search Console
35 GSC_SITE_URL=https://www.bitcot.com
36
37 # Database
38 DATABASE_URL=postgresql://user:password@localhost:5432/bitcot_content
39
40 # Notifications (for breaking news alerts)
41 SLACK_WEBHOOK_URL=https://hooks.slack.com/...
```

Listing 12.1: .env file structure

13. Dashboard Specification

13.1 What to Build

The dashboard is a **React** + **Next.js** web application with a **FastAPI** Python backend. It has three primary views:

13.1.1 View 1: Topic Command Center

This is the main screen a user sees each morning. It shows:

- Breaking news alert (if any spike fired) — red banner at top with “Generate now” button
- 6 ranked topic cards, each showing: topic name, source (Google/X/Reddit), trend sparkline, ICP pre-score, suggested format
- Manual input panel: text area for “your opinion/angle”, image idea input, tone selector (thought leader / founder voice / educational / contrarian)
- Platform toggles: LinkedIn / X thread / Blog / Newsletter
- “Generate all formats” button

13.1.2 View 2: Human Review Panel

Shown after content is generated:

- Three-column layout: blog draft (left) / LinkedIn preview (center) / X thread (right)
- Each column is inline-editable
- Image brief shown below with “Approve image brief” checkbox
- All [NEEDS HUMAN CHECK] flags highlighted in amber
- Approve / Edit / Regenerate section / Reject buttons
- Schedule panel (bottom): time pickers for each platform

13.1.3 View 3: Analytics Dashboard

Weekly view showing:

- 2x2 matrix with all published content plotted
- Top performers (engagement + conversion) highlighted in green
- Underperformers flagged in red with one-click “Optimise” button
- SEO position tracker (pages moving up/down)

- Geo traffic chart (India vs USA split — a key monitoring metric)
- Form conversion funnel (visits → CTA clicks → submissions)
- Memory layer update log (what changed this week)

14. Key Business Rules

14.1 Rules That Must Never Be Violated

1. **Human gate cannot be bypassed.** No content publishes without human approval. Ever. Not even in breaking news mode.
2. **No hallucinated statistics.** If a claim cannot be verified from the source hierarchy, it does not enter the brief. Use [NEEDS HUMAN CHECK] for anything uncertain.
3. **Maximum 1 brand mention per content piece.** The ICP Agent and Writer Agent both enforce this. The Repurposing Agent inherits the same rule.
4. **Blog URL never goes in LinkedIn post body.** Always in first comment. LinkedIn algorithmically suppresses posts with external links.
5. **No topic that scored below 0.50 ICP goes to publication.** The ICP Agent rejects it. The trigger agent finds the next topic.
6. **Anti-cannibalization check always runs.** No similar topics within a 5-day window.
7. **All content must target US enterprise buyers.** The geo mismatch (India getting 2x clicks vs USA) is a known problem. Every ICP scoring decision must account for this.

14.2 Content Quality Thresholds

Metric	Minimum target	Action if below
LinkedIn engagement rate	1.82% (current baseline)	Regenerate hook
LinkedIn CTR	0.8%	Check if URL in body (should be in comment)
Blog organic CTR	1.5%	Rewrite title tag
Form conversion rate	0.5% (target; current 0.26%)	Flag form UX issue
X thread bookmark rate	2%	Improve thread structure
ICP score	≥ 0.65	Reject or reshape

15. Build Order and Dependencies

15.1 Recommended Build Sequence

Build in this order to allow incremental testing:

1. **Memory layer schema** — must exist before any agent can run
2. **Trigger Agent** — test with manual keyword list, no spike detection yet
3. **Research Agent** — test with hardcoded topic input
4. **ICP Alignment Agent** — test with hardcoded brief
5. **SEO Agent** — requires DataForSEO API credentials
6. **Writer Agent** — test with hardcoded ICP + SEO package
7. **Human Review Dashboard** (View 1 + View 2) — wire to Writer Agent output
8. **Repurposing Agent** — test with approved blog post
9. **Scheduler + Publisher** — test in sandbox mode (no real posts yet)
10. **End-to-end Mode B test** — run full pipeline, publish one real post
11. **Analytics Agent** — wire to GA4 + LinkedIn + X APIs
12. **Analytics Dashboard** (View 3)
13. **X spike detection** (Mode A) — last because it requires careful threshold tuning
14. **Breaking news end-to-end test** — simulate a spike, run Mode A

15.2 External Services Checklist

Before starting development, confirm access to:

Service	Access level needed	Cost	Priority
Anthropic API	API key	Pay-per-use	Critical
X API v2	Basic tier ($\geq$ \$100/mo)	\$100/mo	Critical
Reddit PRAW	Free (read-only)	Free	High
DataForSEO API	Standard plan	\$50–200/mo	High
LinkedIn API	Company page admin + OAuth	Free	Critical
WordPress REST API	Admin credentials	Free	Critical
GA4 API	Service account credentials	Free	High
Google Search Console API	Site owner credentials	Free	Medium
Google Trends (pytrends)	None (unofficial scraper)	Free	High

16. Data Privacy and Compliance

16.1 What Data the System Stores

- **Content performance metrics:** impression counts, engagement rates, clicks. No personally identifiable information (PII).
- **Topic scores:** cached trend data from public APIs. No PII.
- **Brand context:** Bitcot's own business information. No PII.
- **Published content log:** text of posts published, with timestamps. No PII.

16.2 No PII Stored

This system does **not** store, process, or log:

- Names of people who liked, commented, or clicked posts
- Email addresses or contact information from form submissions (these stay in Bitcot's CRM)
- IP addresses or device identifiers
- Any data protected under CCPA (California Consumer Privacy Act)

16.3 API Token Security

All API tokens must be stored as environment variables, never in code. Rotate LinkedIn and X access tokens every 60 days. Use a secrets manager (AWS Secrets Manager or HashiCorp Vault) in production.

A. Glossary

ICP	Ideal Client Profile. The defined characteristics of Bitcot’s best potential clients.
Mode A	Breaking news mode. Triggered by a 3x spike in keyword volume.
Mode B	Daily scheduled mode. Runs every morning at 7:00 AM PST.
Engagement rate	$(\text{Reactions} + \text{Comments} + \text{Reposts} + \text{Clicks}) / \text{Impressions}$. Measured as a percentage.
Hook pattern	The structural formula of the first line of a post. Empirically validated against 36 Bitcot posts.
Memory layer	The shared database all agents read from and write to. Contains brand context, performance history, and learnings.
SERP gap	A keyword where competitors rank but their content is thin, outdated, or poorly matched to search intent. Bitcot’s opportunity to outrank.
Repurposing	Converting a blog post into platform-native formats. Not shortening or copy-pasting — full reformatting for each platform’s algorithm.
2x2 matrix	Content classification: High/Low Engagement $\times$ High/Low Conversion.
DM hook	A keyword (e.g., “AGENT COST”) placed in an X thread CTA. Followers DM the keyword to receive a resource, generating a warm lead.

B. Current Baseline Metrics (May 2026)

These are the benchmarks the system must beat. Sourced from Bitcot's actual analytics data collected during this internship.

Metric	Current value	Source
LinkedIn followers	16,729	LinkedIn admin
LinkedIn avg engagement rate	1.82% (36-post avg)	LinkedIn analytics export
LinkedIn best post engagement	4.61% (healthcare AI infra)	LinkedIn analytics export
LinkedIn article views (newsletter)	8,089 in 3 months	LinkedIn analytics
Newsletter subscribers	5,871	LinkedIn analytics
GA4 unique visitors (3 months)	17,134	GA4
GA4 form submissions (3 months)	45	GA4
Form conversion rate	0.26%	Calculated
Form abandonment rate	96.4%	Calculated (1,268 CTA clicks, 45 su
GSC total clicks (3 months)	5,650+	Google Search Console
Top traffic country	India (2,852 clicks)	Google Search Console
USA traffic	1,463 clicks	Google Search Console
USA CTR	0.06%	Google Search Console (critical pro

C. Top Performing Content Patterns

Derived from analysis of all 36 published LinkedIn posts.

Hook pattern	Avg eng rate	Avg CTR	Reposts	Example
Stat in line 1	3.2%	0.9%	0	“70% of healthcare AI fails...”
Reader scenario	3.5% CTR	3.5%	1	“We asked why teams use spreadsh
Contrarian reframe	2.8%	1.7%	1	“Your team isn’t slow. Workflows a
Alarming conditional	3.3%	2.0%	0	“Is your pipeline shipping vulnerab
Vague aspiration	0.6%	0.3%	0	“Good enough software costs you n
Product announcement	1.0%	0.4%	0	“Latest innovations in Google AI S
Generic cost warning	0.9%	0.3%	0	“Overspending on product develop